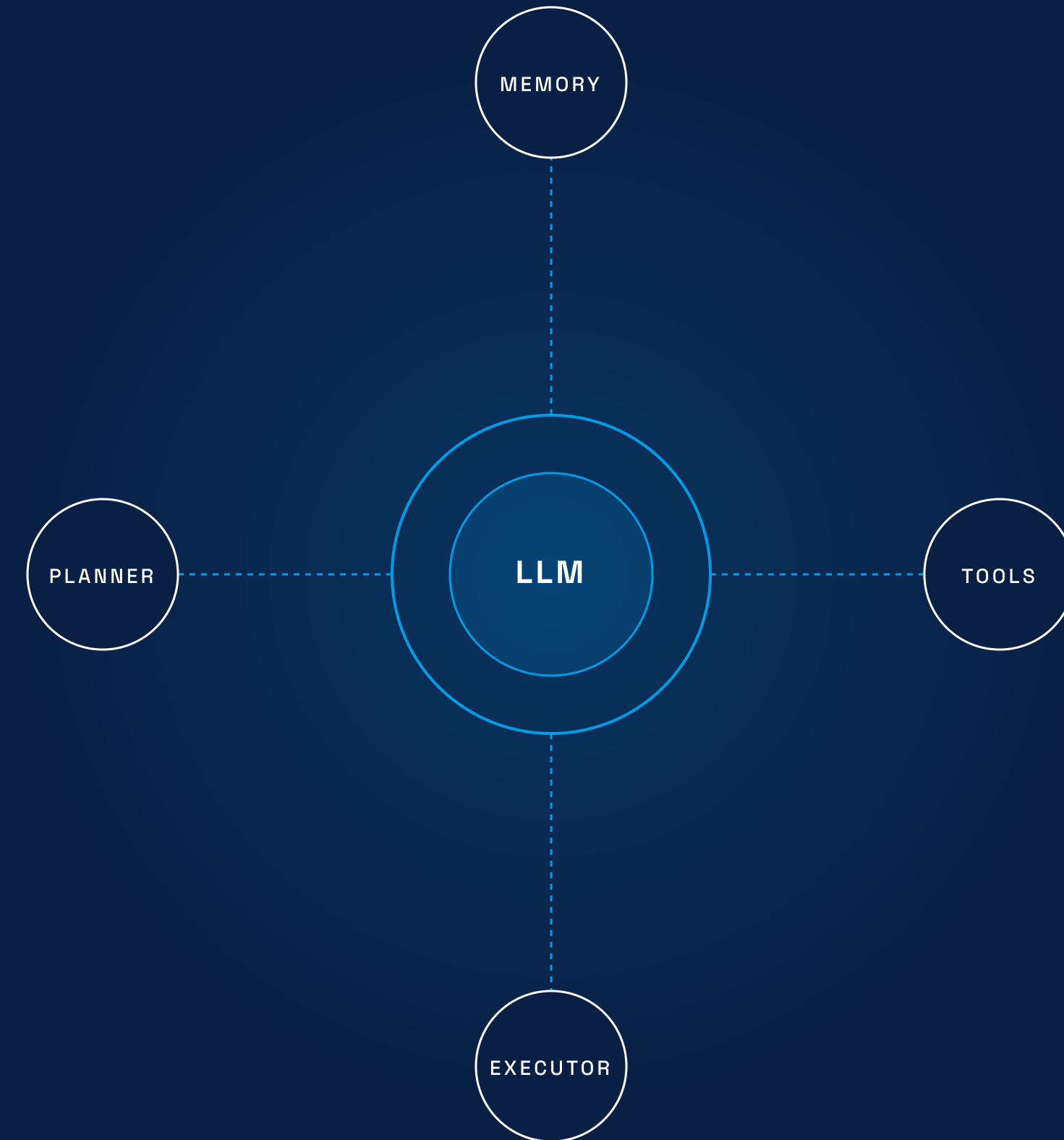


TOPIC 5 • LIVE CORPORATE TRAINING

Agent Architecture.

How agents think, remember & act.

By the end of this session you will draw the anatomy of any agent, trace its reasoning loop, and design a multi-agent system for a real enterprise workflow.



THE PROBLEM

Most “AI agents” in production today are just chatbots in a costume.

CHATBOT

"I'd be happy to help you draft an email about overdue invoices."



 cannot send the email



 forgets you tomorrow



 cannot follow up alone

REAL AGENT

Found 14 overdue invoices. Sent reminders. 1 bounced — escalated to Priya.



 acts on the world



 remembers across sessions



 replans when things break

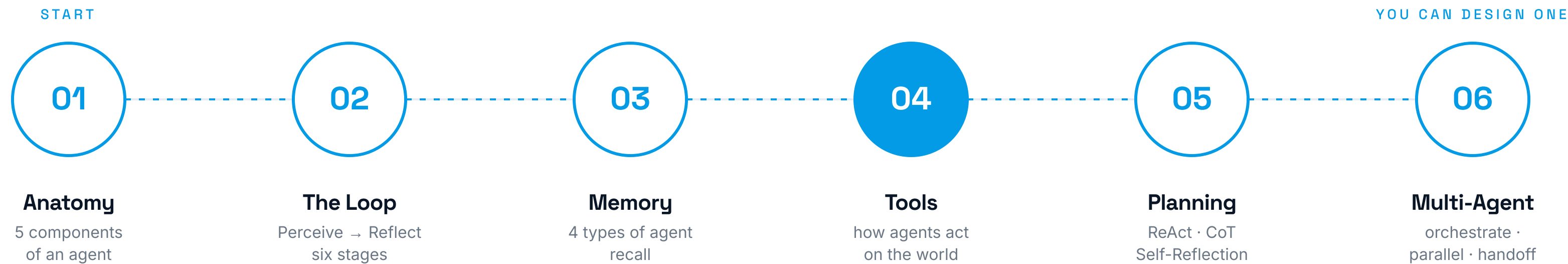
THE GAP

Teams ship “agents” that can talk but cannot act, remember, or recover.

- \$4.4T projected AI agent market by 2030
- 78% of enterprise pilots stall before scaling
- Root cause: missing architecture, not models
- Without architecture, agents drift and fail
- With architecture, they ship and stay shipped

SIX SUB-TOPICS · ONE SESSION

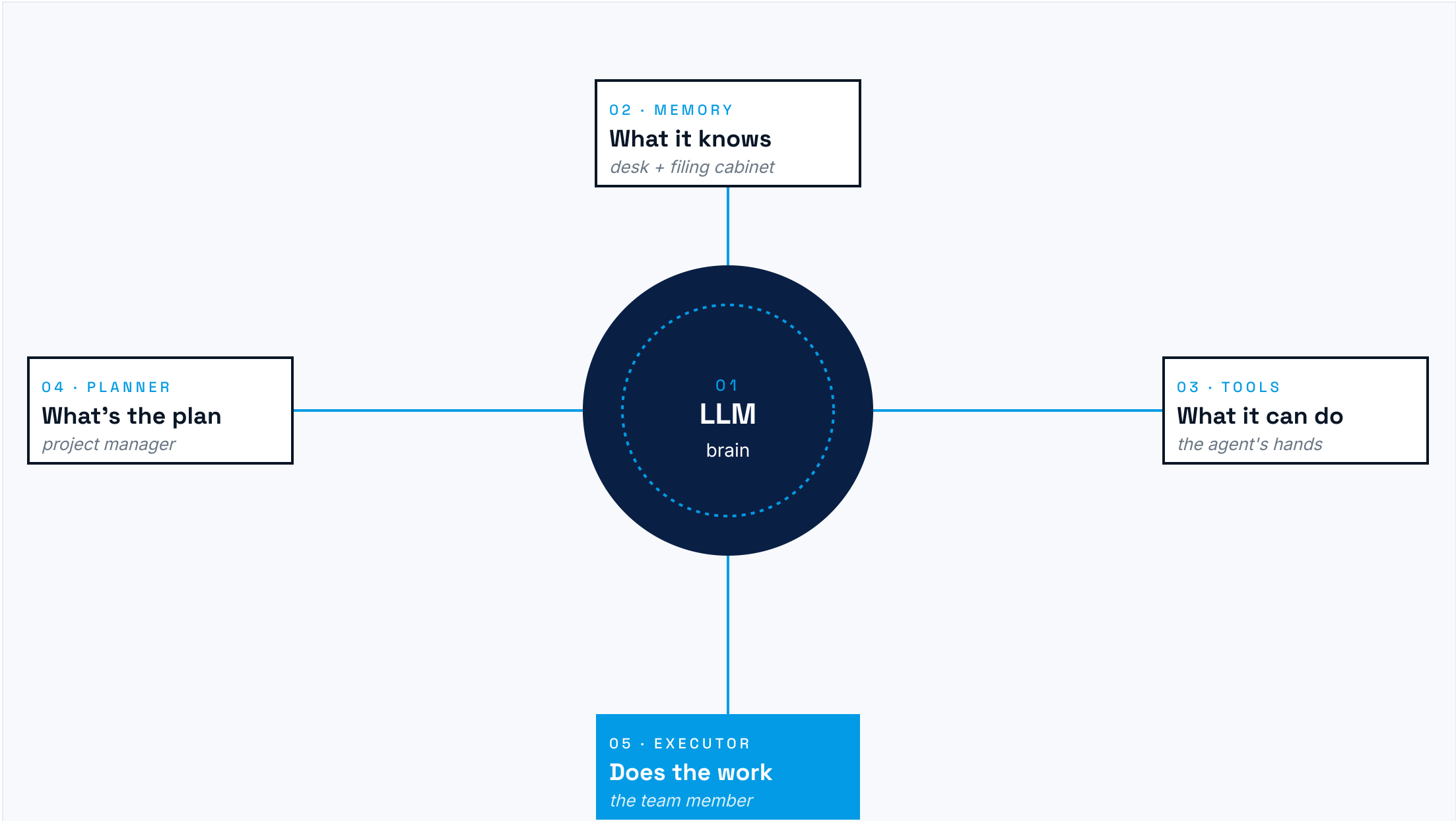
The path from "what is an agent" to "I can design one."



Each sub-topic adds one capability. By slide 16 you will sketch a complete agent for your own business workflow.

FIVE COMPONENTS. ONE MACHINE.

The anatomy of an agent — like a labeled cross-section.



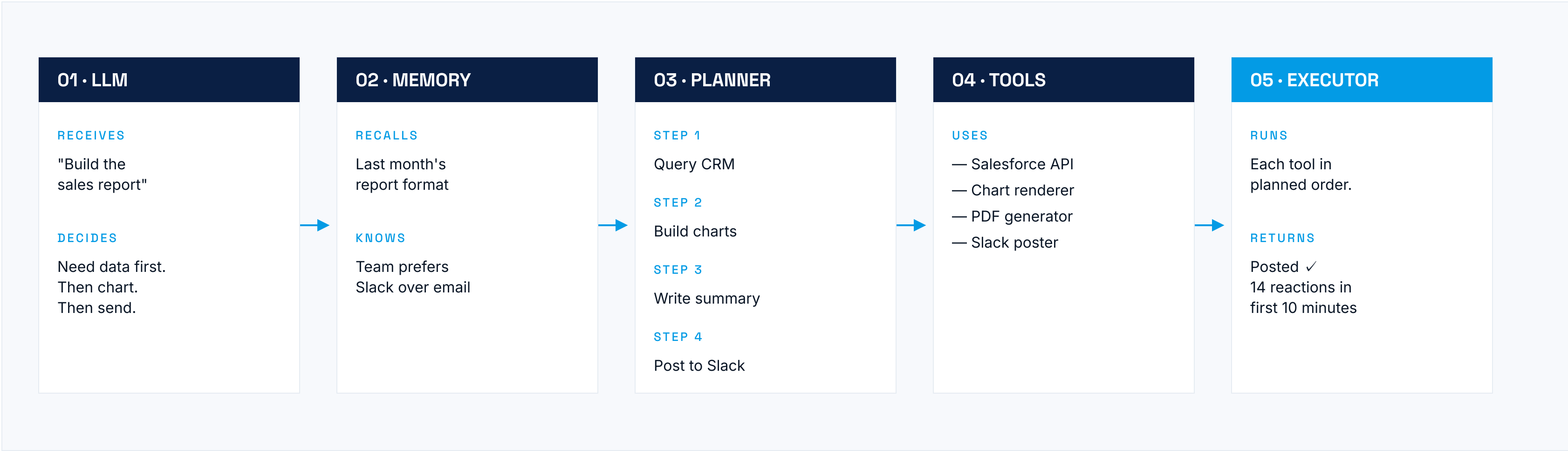
KEY INSIGHT

Remove any one component — the agent breaks.

- No memory → forgets every session
- No tools → can only talk
- No planner → stuck on complex tasks
- No executor → plans never run
- No LLM → no reasoning at all

ONE TASK. FIVE COMPONENTS. WATCH EACH ONE WORK.

”Prepare a monthly sales report and distribute it to the team.”

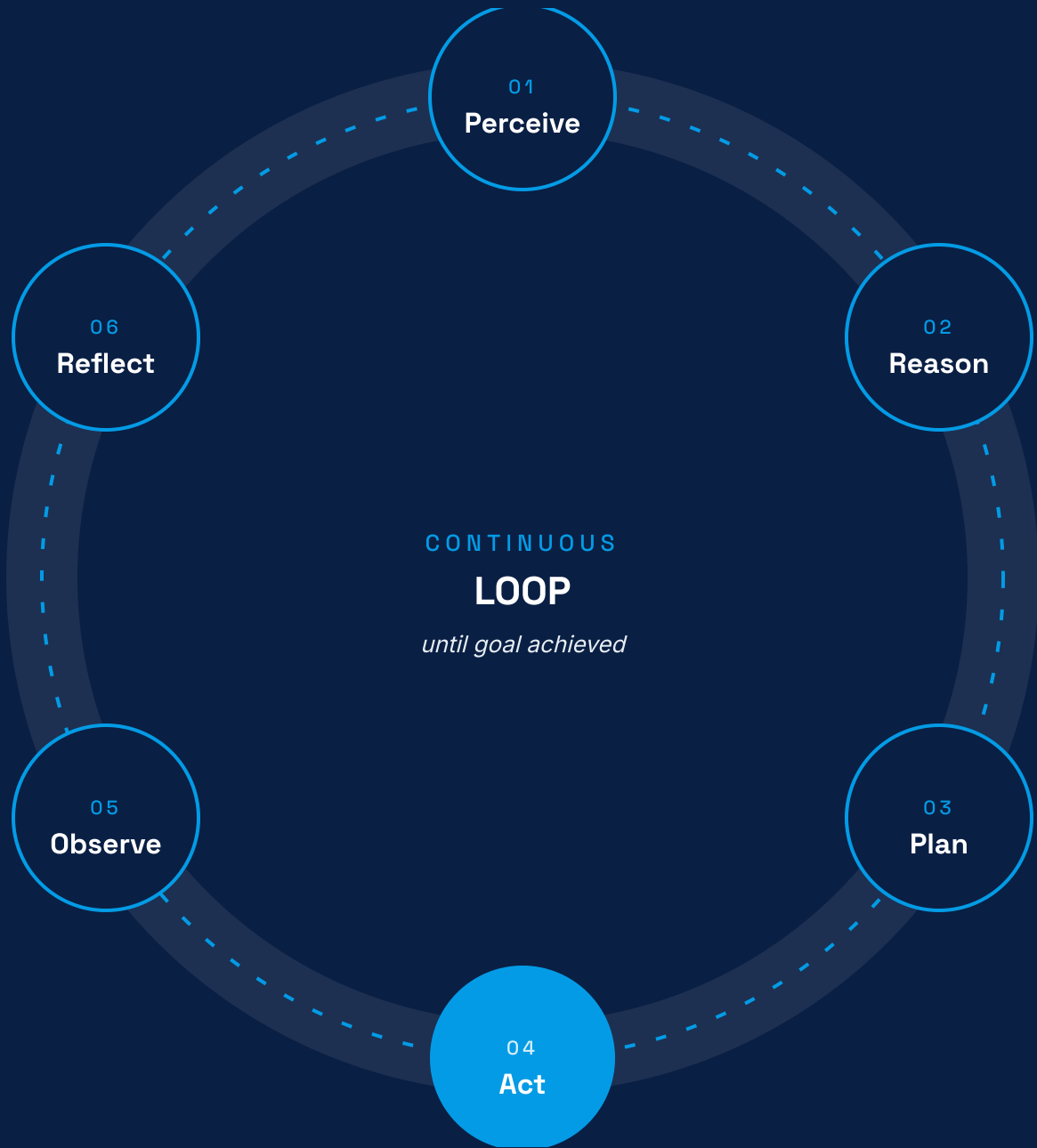


THINK FOR 60 SECONDS

Think of an agent you want to build. Which of the 5 components would be hardest to configure for your use case — and why?

THE REASONING LOOP

Perceive → Reason → Plan → Act → Observe → Reflect.



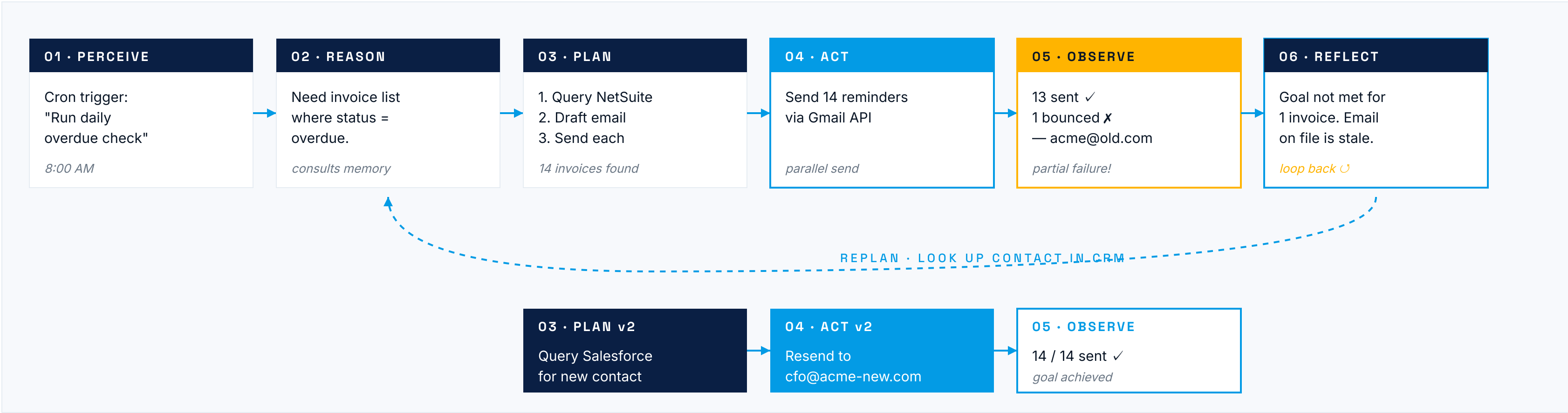
- 01 PERCEIVE | **Receives input** — user message, API trigger, scheduled event, sensor data.
- 02 REASON | **LLM interprets** the input in context of its memory, tools, and goal.
- 03 PLAN | **Decides the sequence** of actions needed to achieve the goal.
- 04 ACT | **Executes first action** — calls a tool, writes to a DB, sends a message.
- 05 OBSERVE | **Receives the result** — success, failure, or partial data.
- 06 REFLECT | **Evaluates progress** — adjusts the plan, loops back to Reason.

ANALOGY

How a great consultant works — understand the brief, think it through, plan, do, review, adjust. Repeat until done.

LIVE TRACE · OVERDUE INVOICE AGENT

Watch the loop replan when an email bounces.



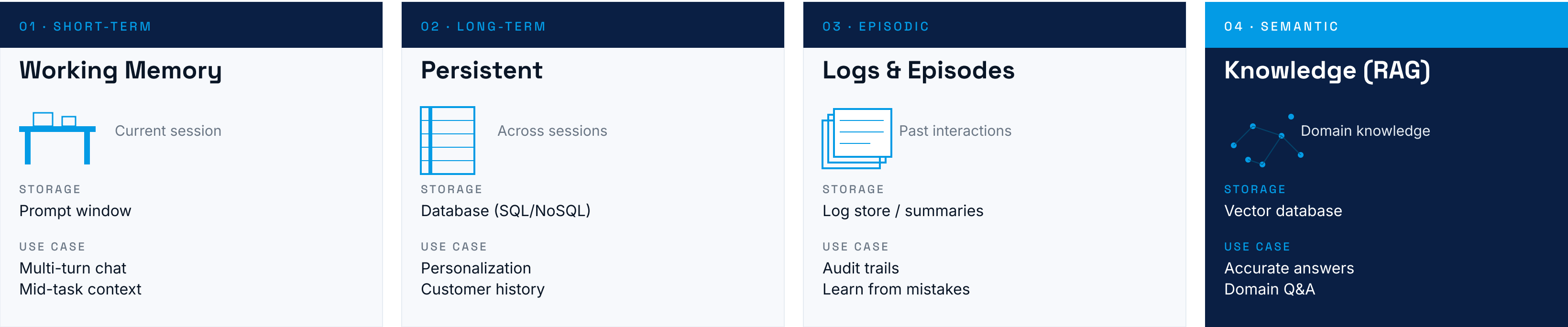
IN PRACTICE — LOOP EXIT CONDITIONS

When should an agent stop looping? Three patterns: goal achieved · max iterations reached · human escalation triggered.

Without exit conditions, agents loop forever and burn tokens. Always set all three. — 07 / 16

FOUR MEMORY TYPES · ONE COHERENT EXPERIENCE

Agents remember in layers — desk, notebook, archive, library.



ANALOGY

Working desk · personal notebook · meeting archive · company library

— Each type solves a distinct problem

— Missing one = broken UX in production

— Match memory type to the business job

WHICH MEMORY DOES YOUR BUSINESS NEED WHERE?

Four memory types × four business domains.

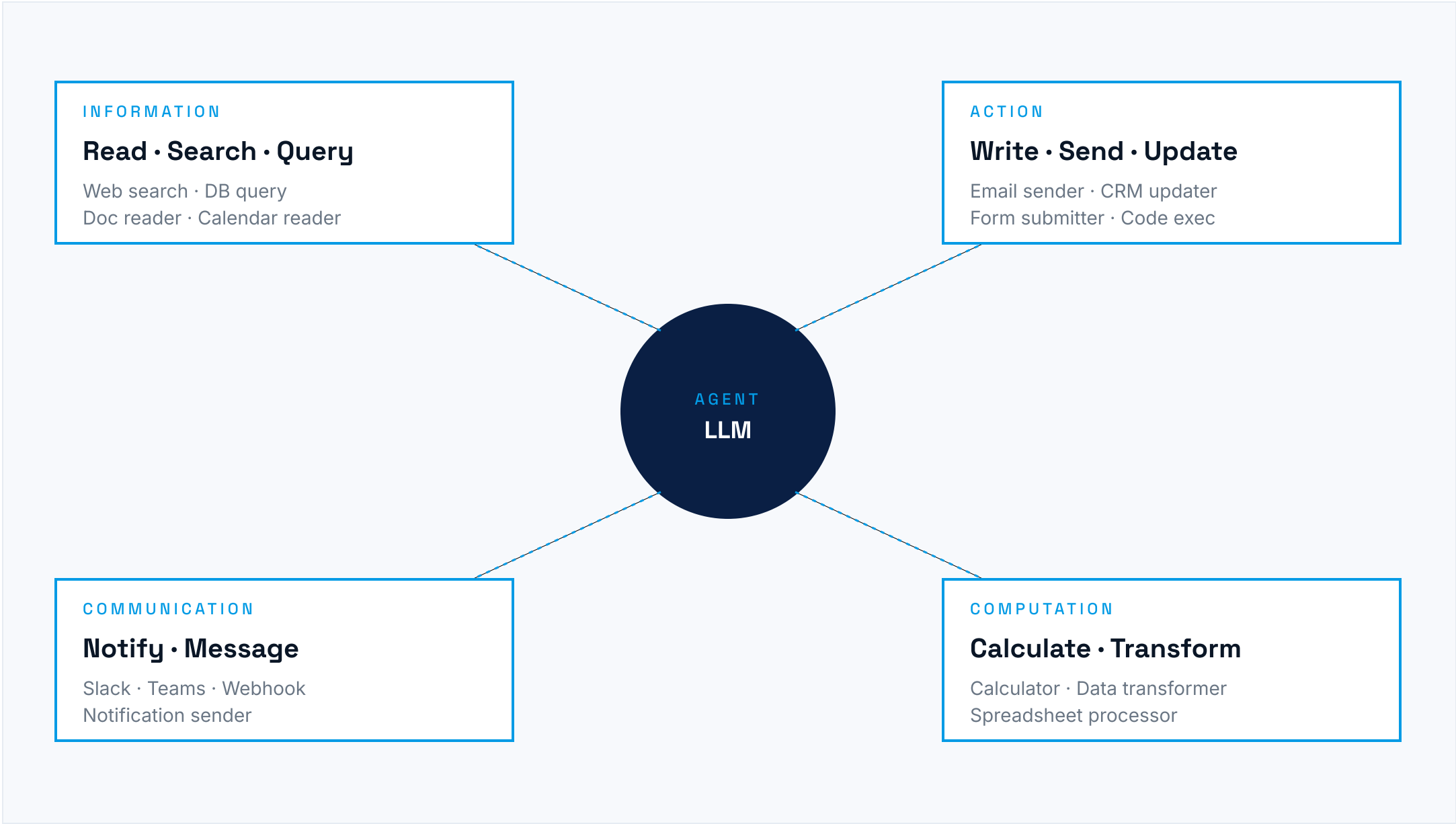
	SHORT-TERM	LONG-TERM	EPISODIC	SEMANTIC
HR recruitment + people ops	Interview Q context in current call live transcript memory	Candidate's full history with company profile DB	Past interview rounds + feedback summaries audit trail	Job descriptions + benefits handbook vector RAG
Sales pipeline + outreach	Current pitch flow with one prospect conversation state	Customer's full buying history CRM	All past calls + outcomes + objections call log	Product catalog + pricing rules vector RAG
Support customer service	Current ticket thread + chat history session state	Customer's product + support tier customer DB	All prior tickets + resolutions ticket archive	Help docs + troubleshoot guides vector RAG
Finance FP&A + reporting	Current analysis variables in flow scratchpad	Vendor master, approval policies ERP	Monthly close logs + prior-period anomalies close history	GAAP rules + company policy doc vector RAG

THINK FOR 60 SECONDS

For the agent you want to build — which memory types does it need? What breaks for the user if each one were removed?

HOW AGENTS ACT ON THE WORLD

Tools are the agent’s hands. The LLM is the mind that picks them up.



HOW TOOL CALLING WORKS

Decide → call → run → receive → reason.

- 1 LLM decides a tool is needed
- 2 Generates structured call (name + params)
- 3 Executor runs the tool
- 4 Result returns to LLM
- 5 LLM uses result in next reasoning step

CRITICAL DESIGN PRINCIPLE

Scope and permission every tool. Broad tool access = security risk.

IN PRACTICE • RECRUITMENT AGENT

The tool manifest defines what the agent can & cannot do.

MANIFEST • RECRUITMENT AGENT

TOOL	SCOPE	PERMISSION
Resume Reader <i>parse PDF • extract fields</i>	Read uploaded files	Read-only
Calendar API <i>schedule interviews</i>	Recruiter's calendar only	Read + Write
Email Sender <i>candidate outreach</i>	Templates only • no free text	Send (templated)
ATS Writer (Greenhouse) <i>log stages + notes</i>	Candidate records only	Write (audited)
Slack Notifier <i>flag hiring managers</i>	#hiring-ops channel only	Post (templated)

WHEN TOOLS FAIL • THE LOOP SAVES YOU

Wrong tool → error → reflect → right tool.

01 • CALL
"Search resume
for Python skill"

02 • ERROR
Called web_search
— wrong tool

03 • REFLECT
Need resume_reader
not web_search

04 • RETRY ✓
Python: 4 yrs
Django • Flask

— Agent didn't crash. It observed the error, replanned, and tried the right tool.

This resilience is why the loop matters.

THREE WAYS AN AGENT CAN THINK

ReAct · Chain-of-Thought · Self-Reflection.

STRATEGY 01

ReAct · Reason + Act

THINK
"What do I know?"

ACT
web_search("...")

THINK
"Now I know X. Need Y."

ACT
db_query("...")

...alternate think/act...

BEST FOR

Tool-heavy tasks with dynamic replanning.
Example: research a topic with iterative web search.

STRATEGY 02

Chain-of-Thought

STEP 1 Vendor A: review criteria 1–6

STEP 2 Vendor B: review criteria 1–6

STEP 3 Vendor C: review criteria 1–6

STEP 4 Compare scores side by side

FINAL ANSWER
"Recommend Vendor B — best fit on cost & SLA"

no tools called · pure reasoning chain

BEST FOR

Complex analysis · multi-variable decisions.
Example: evaluate 3 vendor proposals against 6 criteria.

STRATEGY 03

Self-Reflection Loop

DRAFT
Write contract clause

CRITIQUE
"Term 3 is ambiguous."

CRITIQUE
"Better. Almost done."

REDRAFT
Tighten Term 3

FINALIZE
Quality threshold met · output approved

Loops until quality threshold or max iterations.

BEST FOR

High-stakes outputs · reports · legal · proposals.
Example: draft → critique → redraft a contract clause.

Same task. Three reasoning paths. Choosing the right one is half the design.

SAME QUESTION · THREE STRATEGIES

”Should we expand into the Southeast Asian market?”

REACT

Live research with tools.

Think: What's GDP growth in SEA?

Act: web_search("SEA GDP 2025")

Think: Need competitor data too.

Act: crm_lookup("competitors")

Think: Need our cash runway.

Act: finance_api("runway")

Output: live data-grounded brief

CHAIN-OF-THOUGHT

Pure structured reasoning.

Step 1: Define expansion criteria

Step 2: List SEA markets

Step 3: Score each on 6 criteria

Step 4: Weight by strategic fit

Step 5: Rank & recommend

— Uses memory only, no tools

Output: structured framework decision

SELF-REFLECTION

Draft → critique → redraft.

v1: "Yes, expand to Vietnam."

Critique: "Why not Indonesia?"

v2: "Vietnam first, then Indonesia."

Critique: "Cash for 2 markets?"

v3: "Vietnam Q1 · IDN if hits ARR target"

— Each pass tightens the answer

Output: nuanced board-ready memo

IN PRACTICE · STRATEGY SELECTION GUIDE

Three questions to ask before you choose:

01 · TASK COMPLEXITY

Simple lookup → CoT. Multi-source live data → ReAct. Open-ended creative → Reflection.

02 · TOOL DEPENDENCY

Needs live tools? → ReAct. Internal reasoning only? → CoT or Reflection.

03 · OUTPUT QUALITY

High-stakes deliverable → Reflection. Internal analysis → CoT. Speed matters → ReAct.

WHEN ONE AGENT ISN'T ENOUGH

Three patterns for multi-agent systems.

PATTERN 01

Orchestrator-Worker

```
graph TD; Orchestrator[ORCHESTRATOR  
Planner Agent] --> WorkerA[WORKER A  
Web research]; Orchestrator --> WorkerB[WORKER B  
Financial data]; Orchestrator --> WorkerC[WORKER C  
Competitors]; WorkerA -.-> Final[Final synthesis]; WorkerB -.-> Final; WorkerC -.-> Final;
```

ANALOGY

Project manager delegating to a team.

BEST FOR

Complex tasks with sequential dependencies.

Ex: market research → web + finance + competitor → unified brief.

PATTERN 02

Parallel Agents

```
graph TD; Resumes[500 resumes] --> Agent1[AGENT 1  
Batch 1  
125 resumes]; Resumes --> Agent2[AGENT 2  
Batch 2  
125 resumes]; Resumes --> Agent3[AGENT 3  
Batch 3  
125 resumes]; Resumes --> Agent4[AGENT 4  
Batch 4  
125 resumes]; Agent1 -.-> Shortlist[Ranked shortlist]; Agent2 -.-> Shortlist; Agent3 -.-> Shortlist; Agent4 -.-> Shortlist;
```

ANALOGY

Multiple analysts splitting a shared queue.

BEST FOR

Independent sub-tasks · speed matters.

Ex: process 500 resumes in parallel → merged ranking.

PATTERN 03

Specialist Handoffs

```
graph LR; Ticket[TICKET  
"login broken"] --> Classifier[CLASSIFIER  
→ category]; Classifier --> Resolver[RESOLVER  
try standard fix]; Resolver --> Escalate[ESCALATE  
→ human]; Escalate --> FollowUp[FOLLOW-UP  
closes loop];
```

Each agent handles ONE phase, then hands off to the next.

Like a relay race.

ANALOGY

A relay race — each runner specialized.

BEST FOR

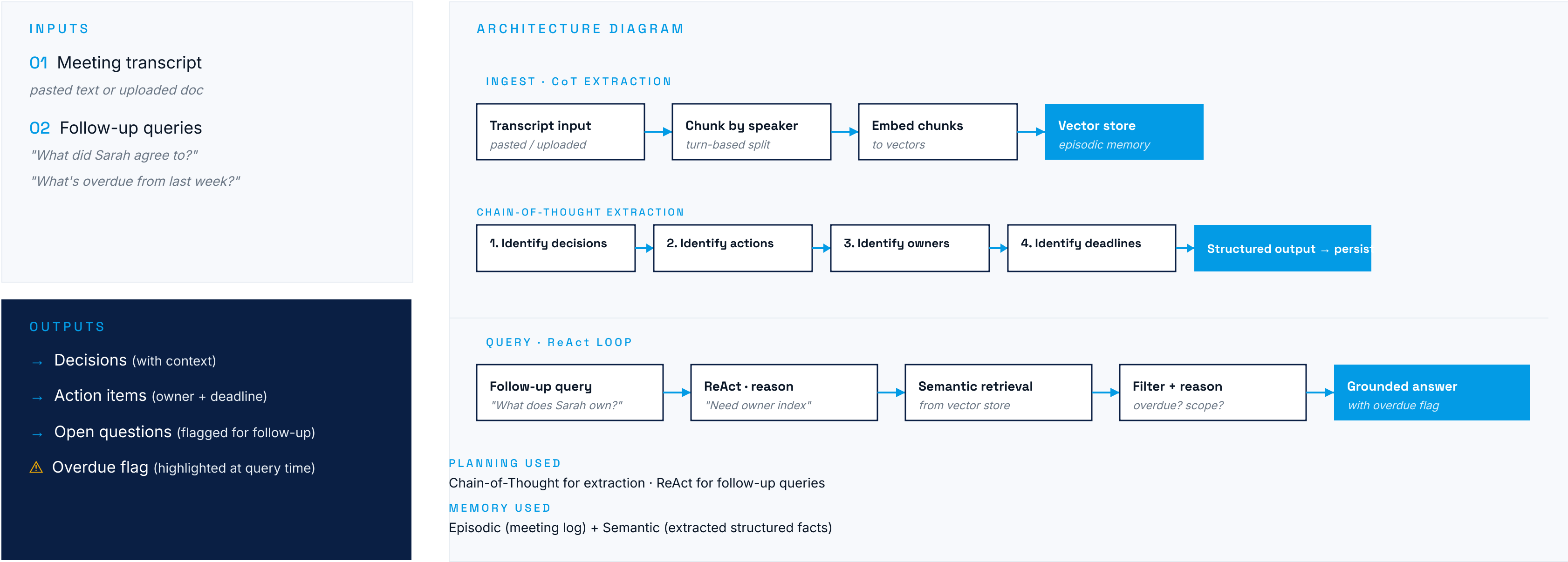
Distinct sequential phases needing different expertise.

Ex: support ticket → classify → resolve → escalate → close.

THE AGENT WE JUST DESIGNED — END TO END

Meeting Intelligence Agent.

Reads any meeting transcript → extracts decisions, action items, owners, deadlines → answers your questions about who owns what.



WHAT YOU CAN DO NOW

Six concepts. One mental model. Yours now.



01 · ANATOMY

5 components

brain · memory · tools · planner · executor



02 · LOOP

Perceive → Reflect

6 stages · until the goal is met



03 · MEMORY

4 types

desk · notebook · archive · library



04 · TOOLS

The agent's hands

information · action · communication · compute



05 · PLANNING

3 strategies

ReAct · Chain-of-Thought · Self-Reflection



06 · MULTI-AGENT

3 patterns

orchestrate · parallel · handoff

WHAT'S NEXT · TOPIC 6

Build & deploy a live agent using EdYoda's Agent Builder.

Bring the system prompt you wrote in Topic 3.

REFLECT BEFORE NEXT SESSION

Sketch your agent's architecture: which components? which memory types? which planning strategy? which tools? multi-agent or solo?

You will build exactly this in Topic 6.